

LSM Week 2 - R assignment

```
# W2Q1a
"
 $\chi^2$  distribution
(a) For each  $i = 1, 2, \dots, 100$  simulate 1000 random values from
Normal distribution with mean 10 and variance 16
by using command rnorm in R. Denote this data for each  $i$  as  $X_i$ 
"

n_simulations <- 100
n_values <- 1000
pop_mean <- 10
pop_variance <- 16
pop_sd <- sqrt(pop_variance)

data_list <- vector("list", n_simulations)

for (i in 1:n_simulations) {
  data_list[[i]] <- rnorm(n_values, mean = pop_mean, sd = pop_sd)
}

print(data_list)
```

```

[73] 14.82411383  8.25692656  6.08850568  2.91158946 16.02407319 12.09869018  9.11633706  6.08796533
[81] 11.28838363  4.25851318  0.22314029 11.41049983 12.05960587  9.35495868 -1.00585766 16.79101461
[89] 9.51592011 10.74794359 10.53154034 17.55671108 7.28080434 11.39285935  8.74642893  6.68521150
[97] 10.82409196  9.89353215 13.13932174  2.59614312 17.86895600 14.02939169  9.85833455  7.04059029
[105] 15.60699291  5.22110448  6.48919156  9.68574809 14.84190931  4.69008264  1.91814675 12.13751086
[113] 11.20804325  9.91288603  8.87405612  6.14373087  9.17289521  4.61148897 15.15695234 10.38130461
[121] 8.24922542 11.27254520 10.87178053 14.41613367  6.50474726 12.97119210  9.34904428  7.71376360
[129] 14.80558651 10.72760995  9.27374166 16.96130291 11.51414330 17.10502764  9.58962270 18.31779132
[137] 10.86678902  4.25773427  7.86064526  8.57996919 14.37538056  7.44050514 15.10041388  3.50730642
[145] 13.08957596 12.96699723  9.67973814  9.35407364  9.09620586 12.85977563 12.63188815 13.24844691
[153] 16.36536046  6.20338464 10.76451479  5.54376856 10.59687043  6.34191049  5.05636576 13.73578948
[161] 3.82553728 11.57322078  5.33909280 10.61051088  8.46966599  9.10321623  5.18174531 13.07237802
[169] 3.92705815  4.38018839  8.39837033  8.36612930 11.71751716 10.73838137 14.88637991  9.75751332
[177] 12.08940953 10.92312955  3.92660029 19.06272670 12.89497318 12.88491186  4.56513507  9.35430749
[185] 11.05453588 10.53091298  9.66080588 15.19420006 17.65124142 14.83194131 17.02436504  1.11534611

```

```
# W2Q1b
"
Calculate the means of each  $X_i$  ( $i = 1, 2, \dots, 100$ ) and
store it in a new vector 'Mi'.
"
M <- sapply(data_list, mean)
print(M)
```

```
r>
r> M <- apply(data_list, mean)
print(M)
[1] 9.968306 10.016925 9.968417 10.018086 9.730942 9.948399 9.935263 10.161085 10.132102 10.024043
[11] 9.920053 10.111844 9.743887 9.851061 9.735719 10.275196 9.784862 10.239459 9.896501 9.806470
[21] 10.116388 9.621400 9.981861 9.976647 10.033332 9.874909 10.138960 9.964013 9.997888 10.172296
[31] 10.038833 10.023914 10.050892 9.939807 10.222154 9.953341 9.759806 10.144734 9.914868 10.056034
[41] 9.809651 10.037139 10.137396 10.136163 9.975087 10.071132 9.845898 10.059162 9.845196 10.112466
[51] 10.195216 10.015283 10.074490 10.003899 9.902283 10.039421 9.891785 9.979634 9.987139 9.856472
[61] 9.869858 9.961647 10.003841 9.970661 10.131990 10.074628 9.952854 10.057174 10.049271 10.031455
[71] 9.893815 10.081587 10.118873 9.938237 10.045103 10.077324 9.962492 10.092637 10.228696 9.859513
[81] 10.160763 9.974879 10.081599 9.922910 10.043882 10.141241 9.812476 9.984564 9.976806 10.033228
[91] 9.924116 9.879072 9.937347 9.939173 9.966189 9.914064 10.106003 10.216778 9.742117 9.930331
```

W2Q1c

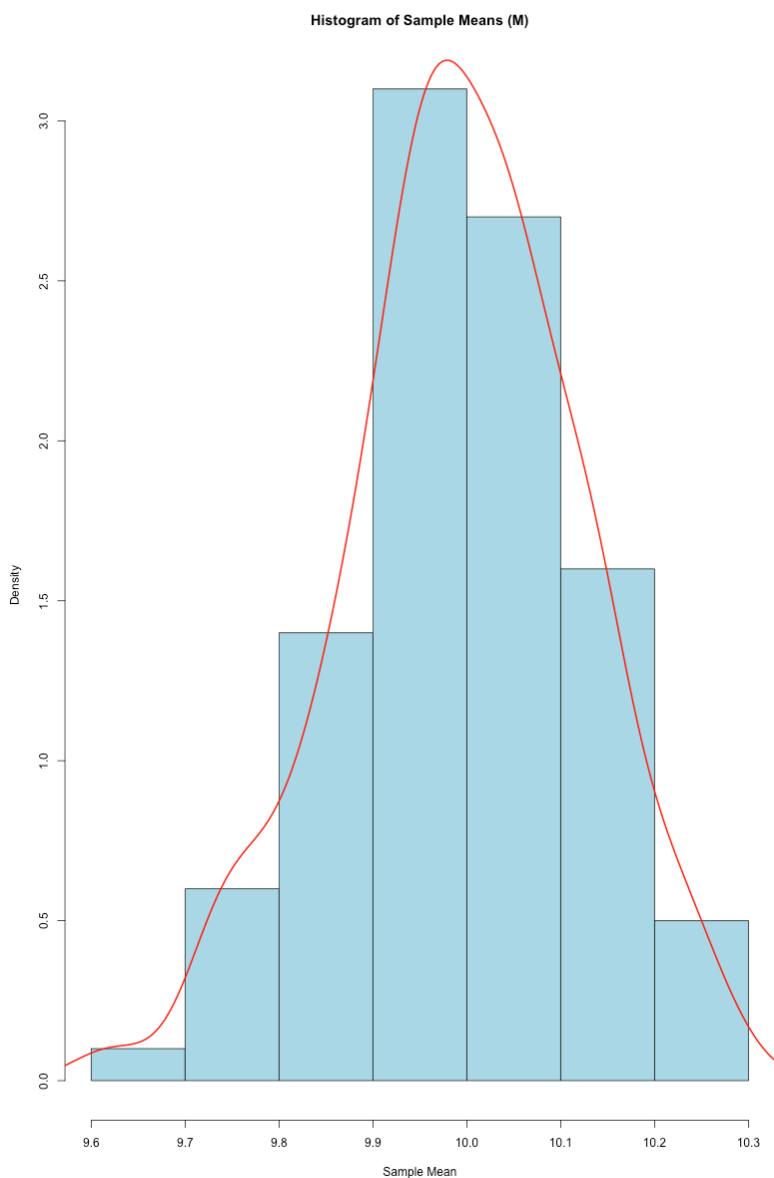
"

Plot the histogram of 'M = {Mi: 1 ≤ i ≤ 100}' using the command hist() in R-software. What should the distribution of M be?

"

```
hist(M,  
     main = "Histogram of Sample Means (M)",  
     xlab = "Sample Mean",  
     col = "lightblue",  
     border = "black",  
     probability = TRUE  
)  
lines(density(M), col = "red", lwd = 2) # Add a density curve
```

According to the Central Limit Theorem (CLT), the distribution of sample means
(M) should be approximately Normal. Since the underlying population is Normal,
the distribution of sample means is exactly Normal.
The theoretical mean of this distribution is the population mean, $\mu = 10$.
The theoretical variance is $\sigma^2/n = 16/1000 = 0.016$.
So, M should follow $N(10, 0.016)$.



```
# W2Q1d
```

```
"
```

```
Calculate the sample variance  $S_i^2$   
of each  $X_i(i = 1, 2, \dots, 100)$ .
```

```
"
```

```
S_sq <- sapply(data_list, var)  
print(S_sq)
```

```
r$> # W2Q1d  
"  
Calculate the sample variance  $S_i^2$   
of each  $X_i(i = 1, 2, \dots, 100)$ .  
"  
S_sq <- sapply(data_list, var)  
print(S_sq)  
[1] "\nCalculate the sample variance  $S_i^2$ \nof each  $X_i(i = 1, 2, \dots, 100)$ .\n"  
[1] 16.28718 15.23000 16.18964 15.05526 14.99710 16.20070 16.56033 17.18580 15.10976 16.14091 15.77365  
[12] 16.05863 15.89028 15.80412 15.98619 15.83458 17.11875 15.35016 16.00539 14.98390 16.16554 17.08190  
[23] 17.32154 16.12813 15.03118 15.12390 16.03609 16.51746 15.80626 13.84197 15.69746 16.07564 16.51002  
[34] 15.61648 16.44034 15.40487 16.92458 15.97369 15.96795 18.13858 15.62190 16.76329 17.42214 15.44740  
[45] 15.35511 16.71108 15.09249 16.48198 15.14024 15.46364 15.09860 15.74463 14.90958 16.78874 16.39129  
[56] 15.94863 16.97483 15.70810 16.35608 16.65306 15.71700 15.12470 15.97349 16.31708 15.76076 15.50080  
[67] 15.68450 16.04662 15.42882 16.19498 16.61064 16.45973 16.41014 17.12421 15.37423 15.40633 16.98564  
[78] 16.91844 14.95220 15.24552 15.88808 17.07143 17.46635 16.03674 15.94658 15.94677 15.77370 15.11038  
[89] 16.56541 17.75272 16.39524 14.87625 16.29724 16.05202 16.60565 15.53906 15.56136 15.56909 16.21925  
[100] 15.86348
```

```
# W2Q1e
```

```
"
```

```
Plot the histogram of 'CHISQ=  $\{(999/16)S_i^2: 1 \leq i \leq 100\}$ '  
using the command hist() in R-software.
```

```
What should the distribution of CHISQ be?
```

```
"
```

```
CHISQ <- (n_values - 1) / pop_variance * S_sq
```

```
hist(CHISQ,
```

```
  main = "Histogram of the Chi-Square Statistic",
```

```
  xlab = "Value of  $(n-1)S^2 / \sigma^2$ ",
```

```
  col = "lightgreen",
```

```
  border = "black",
```

```
  probability = TRUE,
```

```
  breaks = 20
```

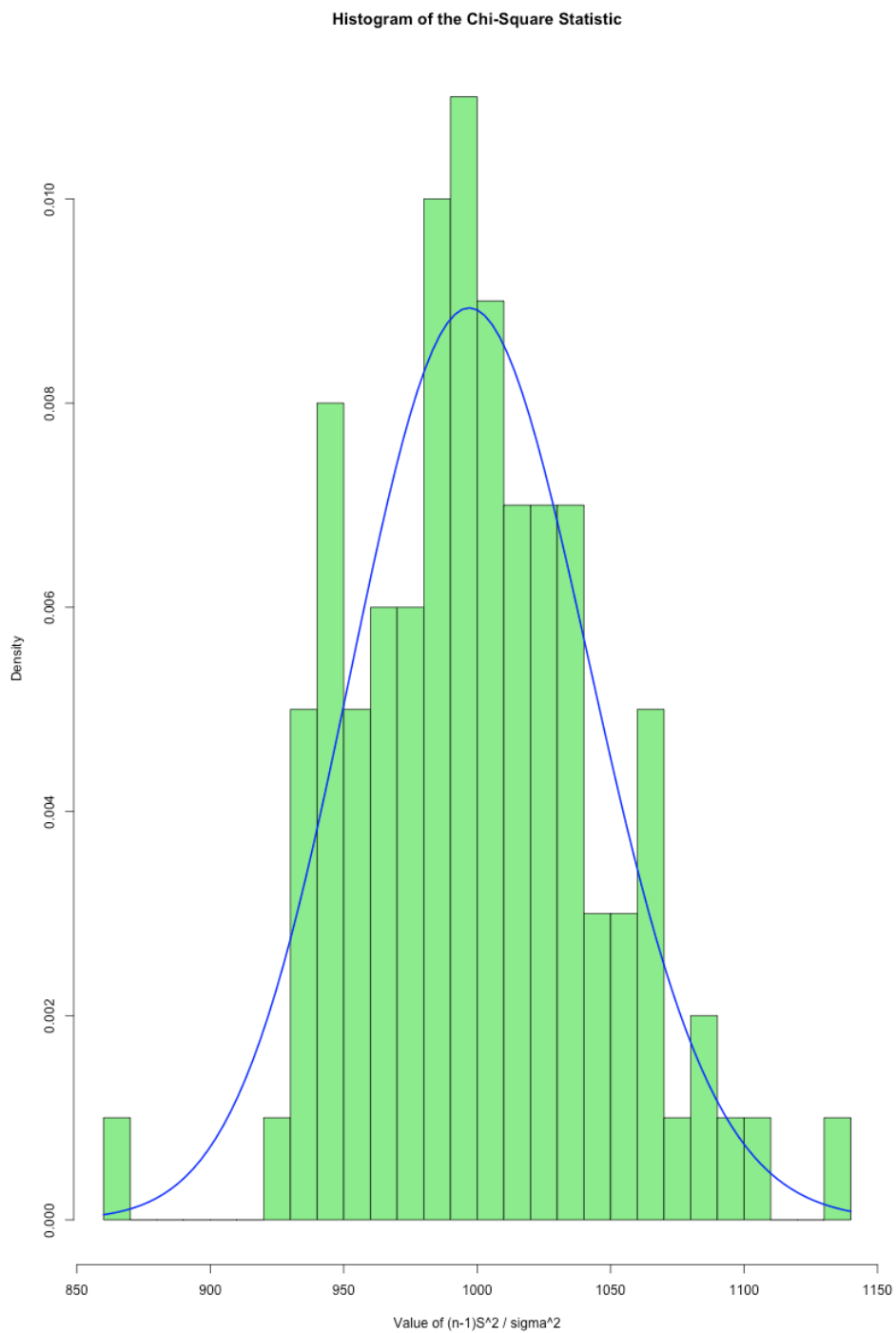
```
)
```

```
# Overlay the theoretical chi-square density curve
```

```
curve(dchisq(x, df = n_values - 1), add = TRUE, col = "blue", lwd = 2)
```

```
# the CHISQ vector should follow a Chi-squared
```

```
# distribution with 999 degrees of freedom.
```



```
# W2Q1f
```

```
"
```

```
Calculate the statistic  $T_i = (M_i - 10)/(S_i/\sqrt{1000})$ 
```

```
for each  $i = 1, 2, \dots, 100$ .
```

```
"
```

```
S <- sqrt(S_sq)
```

```
T_stat <- (M - pop_mean) / (S / sqrt(n_values))
```

```
print(T_stat)
```

```

r$> # W2Q1f
"
Calculate the statistic  $T_i = (M_i - 10)/(S_i/\sqrt{1000})$ 
for each  $i = 1, 2, \dots, 100$ .
"

S <- sqrt(S_sq)
T_stat <- (M - pop_mean) / (S / sqrt(n_values))
print(T_stat)
[1] "\nCalculate the statistic  $T_i = (M_i - 10)/(S_i/\sqrt{1000})$ \nfor each  $i = 1, 2, \dots, 100$ .\n"
[1] -0.24834301 0.13714735 -0.24821645 0.14740200 -2.19706405 -0.40540646 -0.50305931 1.22876673
[9] 1.07468311 0.18924848 -0.63655603 0.88259105 -2.03172939 -1.18473975 -2.09022781 2.18695298
[17] -1.64429846 1.93274528 -0.81809542 -1.58101165 0.91540100 -2.89676014 -0.13781951 -0.18388351
[25] 0.27187528 -1.01716867 1.09733744 -0.28001234 -0.01679829 1.46445783 0.30994886 0.18861252
[33] 0.39607011 -0.48167295 1.73259972 -0.37592722 -1.84630709 1.14516725 -0.67370195 0.41605070
[41] -1.52294271 0.28684701 1.04093124 1.09554897 -0.20104459 0.55025108 -1.25437879 0.46082589
[49] -1.25809953 0.90440931 1.58871610 0.12180029 0.61004803 0.03008771 -0.76324256 0.31215561
[57] -0.83058580 -0.16249937 -0.10056336 -1.11221985 -1.03808406 -0.31186112 0.03038740 -0.22967858
[65] 1.05136562 0.59941039 -0.37644875 0.45134007 0.39666859 0.24717359 -0.82388078 0.63592961
[73] 0.92795748 -0.47197944 0.36375612 0.62296692 -0.28779162 0.71220023 1.87028010 -1.13779568
[81] 1.27541494 -0.19226795 0.61742402 -0.60875106 0.34749727 1.11846709 -1.49310608 -0.12557126
[89] -0.18020832 0.24938317 -0.59263670 -0.99147481 -0.49077891 -0.48010357 -0.26237651 -0.68938317
[97] 0.84975562 1.73733113 -2.02491416 -0.55314980

```

W2W1g

"

Plot the histogram of ' $T = \{T_i: i = 1, 2, 3, \dots, 100\}$ '
using the command `hist()` in R-software.

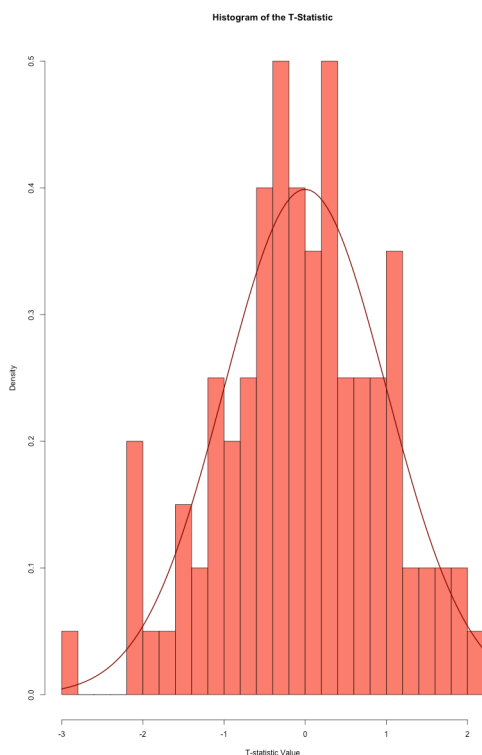
What should the distribution of T be?

"

```

hist(T_stat,
     main = "Histogram of the T-Statistic",
     xlab = "T-statistic Value",
     col = "salmon",
     border = "black",
     probability = TRUE,
     breaks = 20
)
# Overlay the theoretical t-distribution curve
curve(dt(x, df = n_values - 1), add = TRUE, col = "darkred", lwd = 2)
# T should follow a t-distribution with 999 degrees of freedom.

```



```
# W2Q2a
```

```
"
```

Give a brief description of the dataset and
read the data set as data frame (name it 'df') in R.

```
"
```

```
data("faithful")  
str(faithful)
```

```
df <- faithful
```

```
r$> # W2Q2a  
"  
  Give a brief description of the dataset and  
  read the data set as data frame (name it 'df') in R.  
"  
data("faithful")  
str(faithful)  
  
df <- faithful  
[1] "\nGive a brief description of the dataset and\nread the data set as data frame (name it 'df') in R.\n"  
'data.frame': 272 obs. of 2 variables:  
 $ eruptions: num 3.6 1.8 3.33 2.28 4.53 ...  
 $ waiting : num 79 54 74 62 85 55 88 85 51 85 ...
```

```
# W2Q2b
```

```
"
```

Using 'ggplot' in R, plot the scatter plot between the variables
'waiting' and 'eruptions'. On visualizing the scatter-plot, c
omment on the relationship between 'waiting' and 'eruptions'.

```
"
```

```
ggplot(df, aes(x = eruptions, y = waiting)) +  
  geom_point(alpha = 0.6, color = "blue") +  
  labs(  
    title = "Scatter Plot of Waiting Time vs. Eruption Duration",  
    subtitle = "Old Faithful Geyser Data",  
    x = "Eruption Duration (minutes)",  
    y = "Waiting Time to Next Eruption (minutes)"  
  ) +  
  theme_minimal()
```

```
# The scatter plot shows a positive linear relationship between the duration of  
# an eruption and the waiting time until the next eruption. This means that  
# longer eruptions tend to be followed by longer waiting periods. The data also  
# appears to be clustered into two distinct groups, suggesting that a simple  
# linear model might not capture the full complexity of the relationship.
```

Scatter Plot of Waiting Time vs. Eruption Duration
Old Faithful Geyser Data



```
# W2Q2c
```

```
# First, create the base scatter plot
```

```
p_base <- ggplot(df, aes(x = eruptions, y = waiting)) +  
  geom_point(alpha = 0.4, color = "black") +  
  labs(  
    title = "200 Random Linear Models on the Faithful Dataset",  
    x = "Eruption Duration (minutes)",  
    y = "Waiting Time to Next Eruption (minutes)"  
  )
```

```
# Initialize a data frame to store model parameters and their MSE
```

```
random_models <- data.frame(  
  intercept = numeric(200),  
  slope = numeric(200),  
  mse = numeric(200)  
)
```

```
# Generate 200 random models
```

```
for (i in 1:200) {  
  # Generate random intercept and slope from a plausible range  
  # Based on the plot, intercept is ~30-40, slope is ~10  
  b0 <- runif(1, min = 20, max = 50)  
  b1 <- runif(1, min = 5, max = 15)
```

```
  # Calculate predicted values and Mean Squared Error (MSE)
```

```
  predictions <- b0 + b1 * df$eruptions  
  mse <- mean((df$waiting - predictions)^2)
```

```
  # Store the results
```

```
  random_models[i, ] <- c(b0, b1, mse)
```

```
  # Add the line to the plot
```

```
  p_base <- p_base + geom_abline(intercept = b0, slope = b1, color = "gray", alpha = 0.5)  
}
```

```
# Print the plot with all 200 random lines
```

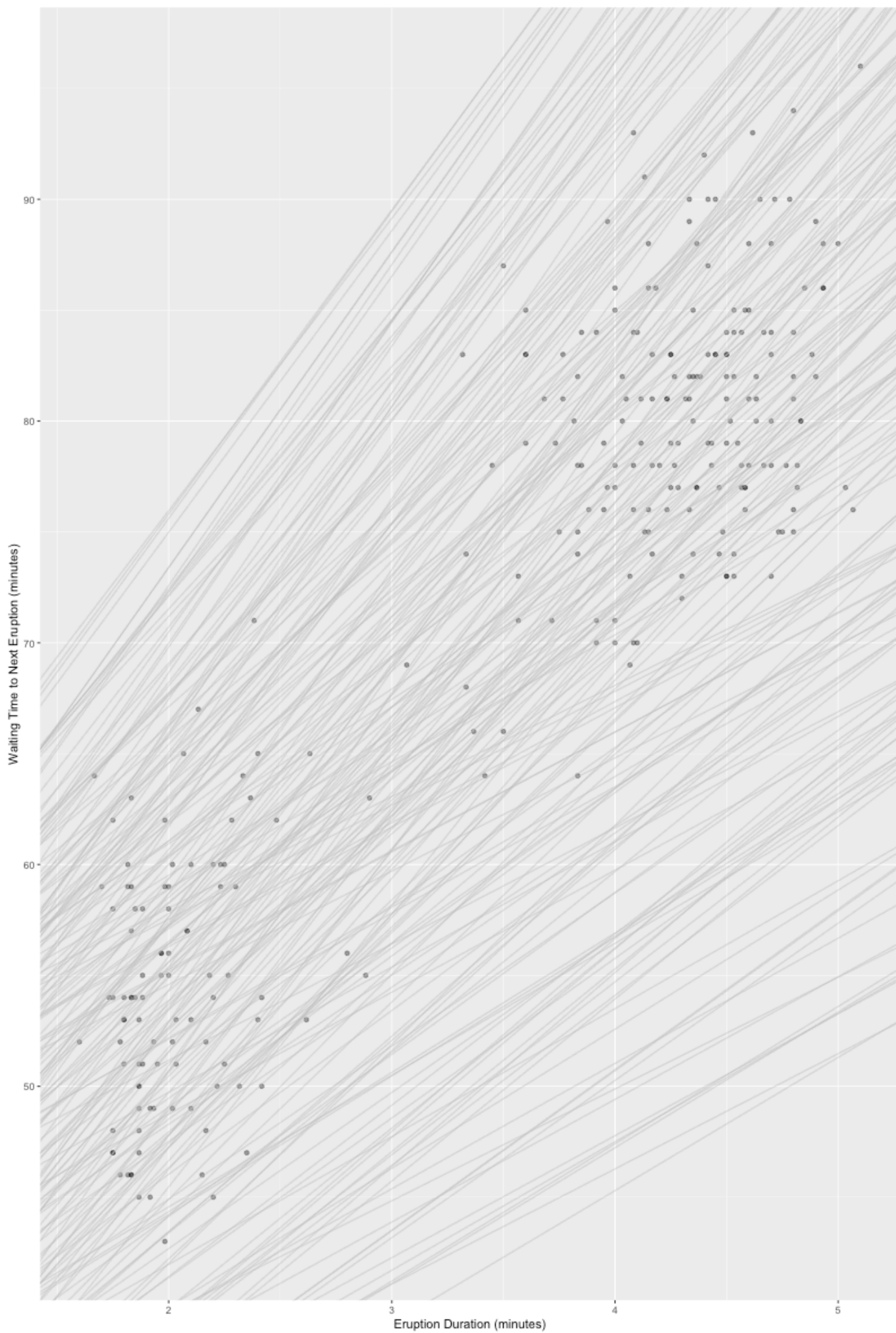
```
print(p_base)
```

```
# The `random_models` dataframe now contains the MSE for each model
```

```
head(random_models)
```

```
}  
  
# Print the plot with all 200 random lines  
print(p_base)  
  
# The `random_models` dataframe now contains the MSE for each model  
head(random_models)  
intercept    slope      mse  
1  31.09337  6.905745 300.75088  
2  47.92091  6.509671  57.90611  
3  44.93441  6.429117  71.24960  
4  40.69733  5.239675 216.04052  
5  31.69843  6.256057 362.71869  
6  24.57575 11.600908  70.04148
```


200 Random Linear Models on the Faithful Dataset



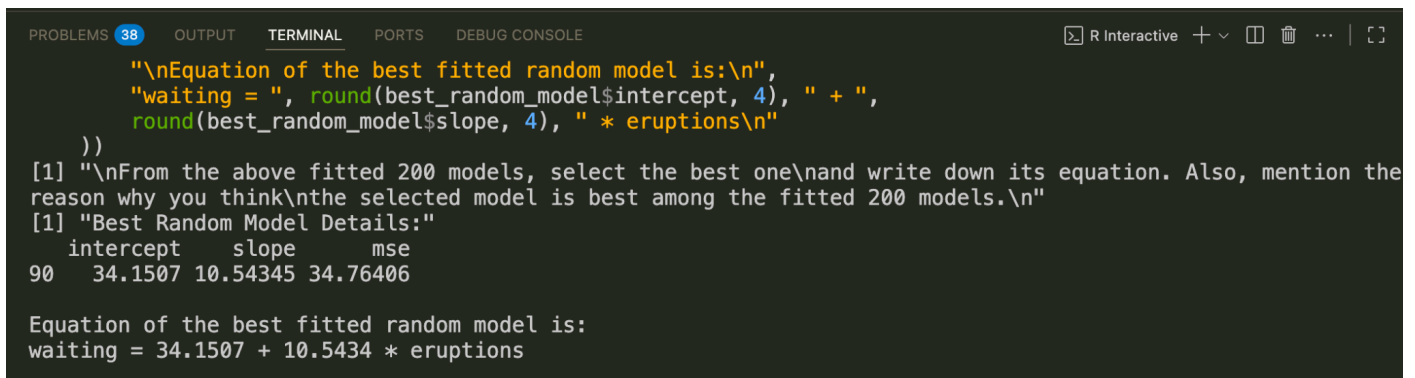
```
# W2Q2d
"

From the above fitted 200 models, select the best one
and write down its equation. Also, mention the reason why you think
the selected model is best among the fitted 200 models.
"

# The best model is the one that minimizes the Mean Squared Error (MSE).
best_random_model <- random_models[which.min(random_models$mse), ]

# Print the details of the best model
print("Best Random Model Details:")
print(best_random_model)

# Write down its equation
cat(paste0(
  "\nEquation of the best fitted random model is:\n",
  "waiting = ", round(best_random_model$intercept, 4), " + ",
  round(best_random_model$slope, 4), " * eruptions\n"
))
```



The screenshot shows an R Interactive terminal window with the following content:

```

PROBLEMS 38 OUTPUT TERMINAL PORTS DEBUG CONSOLE
"\nEquation of the best fitted random model is:\n",
"waiting = ", round(best_random_model$intercept, 4), " + ",
round(best_random_model$slope, 4), " * eruptions\n"
))
[1] "\nFrom the above fitted 200 models, select the best one\nand write down its equation. Also, mention the
reason why you think\nthe selected model is best among the fitted 200 models.\n"
[1] "Best Random Model Details:"
      intercept      slope      mse
90    34.1507 10.54345 34.76406

Equation of the best fitted random model is:
waiting = 34.1507 + 10.5434 * eruptions

```

```
# W2Q2e
"

Use the inbuilt function lm() in R, to find the equation of
best fitted linear model ('waiting' on 'eruptions') for the 'faithful' dataset.
"

model_lm <- lm(waiting ~ eruptions, data = df)

# Get the coefficients
lm_coefs <- coef(model_lm)

# Print the coefficients
print("Coefficients from the lm() function:")
print(lm_coefs)

# Write the equation of the best-fitted model from lm()
cat(paste0(
  "\nEquation of the best fitted model using lm() is:\n",
  "waiting = ", round(lm_coefs[1], 4), " + ",
```

```
round(lm_coefs[2], 4), " * eruptions\n"
))
```

The screenshot shows an R Interactive terminal window with the following content:

```
round(lm_coefs[2], 4), " * eruptions\n"
))
```

Output:

```
[1] "\nUse the inbuilt function lm() in R, to find the equation of\nbest fitted linear model ('waiting' on 'e
rptions') for the 'faithful' dataset.\n"
[1] "Coefficients from the lm() function:"
(Intercept)    eruptions
   33.47440     10.72964
```

Equation of the best fitted model using lm() is:
waiting = 33.4744 + 10.7296 * eruptions

```
# W2Q2f
```

```
"
```

Using 'ggplot' in R, plot the linear models obtained in part (d) and (e) in the same scatter plot (plotted in part (b)).

```
"
```

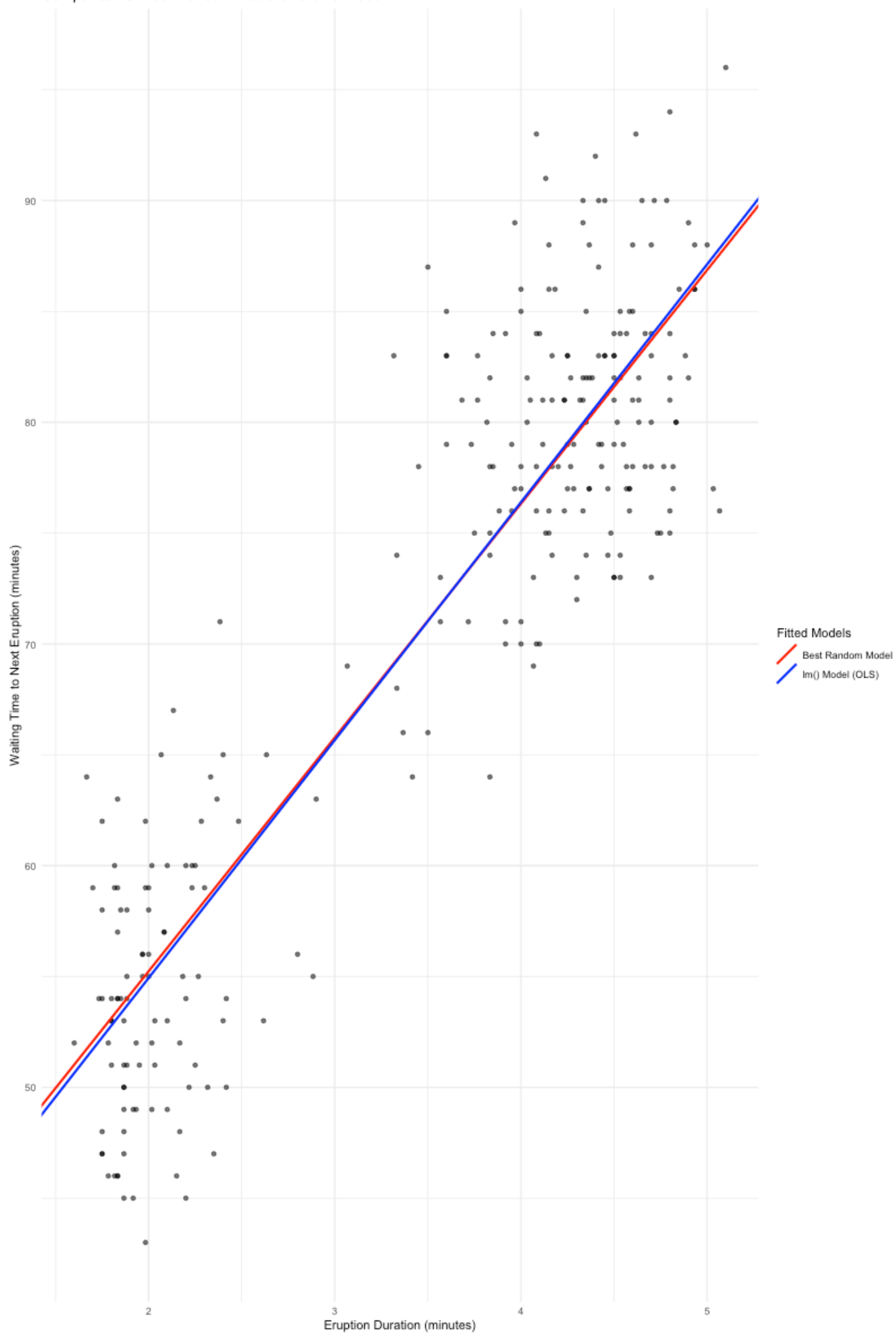
```
# Create a data frame containing the parameters for both lines
```

```
line_models <- data.frame(
  model_type = c("Best Random Model", "lm() Model (OLS)"),
  intercept = c(best_random_model$intercept, lm_coefs[1]),
  slope = c(best_random_model$slope, lm_coefs[2])
)
```

```
# Now, create the plot using this new data frame for the ablines
```

```
ggplot(df, aes(x = eruptions, y = waiting)) +
  geom_point(alpha = 0.6, color = "black") +
  labs(
    title = "Comparison of Best Random Model and OLS Model",
    x = "Eruption Duration (minutes)",
    y = "Waiting Time to Next Eruption (minutes)"
  ) +
  # Use the new data frame and map aesthetics for intercept, slope, and color
  geom_abline(
    data = line_models,
    aes(intercept = intercept, slope = slope, color = model_type),
    linewidth = 1
  ) +
  # The manual scale now works perfectly with the 'model_type' variable
  scale_color_manual(
    name = "Fitted Models",
    values = c("Best Random Model" = "red", "lm() Model (OLS)" = "blue")
  ) +
  theme_minimal()
```

Comparison of Best Random Model and OLS Model



```
# W2Q2g
```

```
"
```

```
For each of the fitted models (part (d) and (e)),  
plot the 'residuals' vs 'eruptions' using 'ggplot' in R.  
Comment on the obtained plots.
```

```
"
```

```
# 1. Residual plot for the best random model from part (d)
```

```
df$residuals_random <- df$waiting - (best_random_model$intercept + best_random_model$slope *  
df$eruptions)
```

```
plot_random_residuals <- ggplot(df, aes(x = eruptions, y = residuals_random)) +  
  geom_point(alpha = 0.6) +  
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +  
  labs(  
    title = "Residuals vs. Eruptions (Best Random Model)",  
    x = "Eruption Duration (minutes)",  
    y = "Residuals"  
  ) +  
  theme_minimal()
```

```
print(plot_random_residuals)
```

```
# 2. Residual plot for the lm() model from part (e)
```

```
df$residuals_lm <- residuals(model_lm)
```

```
plot_lm_residuals <- ggplot(df, aes(x = eruptions, y = residuals_lm)) +  
  geom_point(alpha = 0.6) +  
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +  
  labs(  
    title = "Residuals vs. Eruptions (lm() Model)",  
    x = "Eruption Duration (minutes)",  
    y = "Residuals"  
  ) +  
  theme_minimal()
```

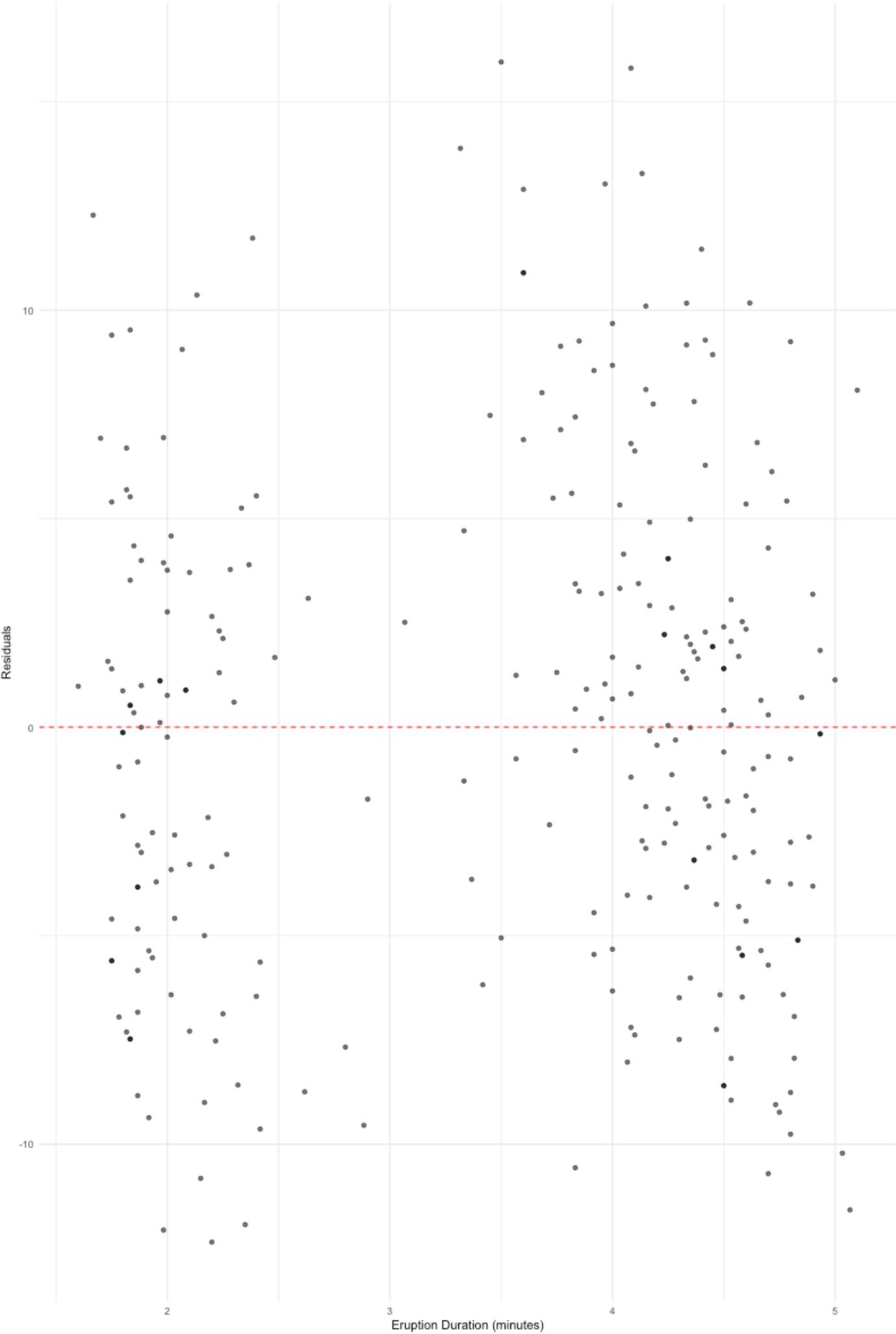
```
print(plot_lm_residuals)
```

```
# Both residual plots — one from the Best Random Model and the other from the lm() model — display a  
# very similar pattern. The residuals are spread around the horizontal line at zero, but they show a distinct  
# clustering pattern for shorter and longer eruption durations (around 2 and 4–5 minutes respectively). This  
# indicates that the residuals are not randomly scattered and there is non-linearity present in the  
# relationship between eruption duration and waiting time.
```

```
# Conclusion:
```

```
# Residuals are centred around zero (no major bias).  
# Variance appears roughly constant.  
# Clear clustering pattern suggests a non-linear relationship between waiting time and eruption duration.  
# The linear assumption of lm() is partially violated.
```

Residuals vs. Eruptions (Best Random Model)



Residuals vs. Eruptions (lm() Model)

